

(Routing) الدرس 2: أساسيات التوجيه

جدول المحتويات

1. مقدمة في نظام التوجيه
2. HTTP أنواع المسارات
3. معاملات المسار
4. المسارات المسماة
5. مجموعات المسارات
6. Route Model Binding
7. التمارين العملية

مقدمة في نظام التوجيه

هو بوابة التطبيق. كل طلب يصل إلى تطبيقك يمر عبر نظام التوجيه الذي يقرر كيفية معالجته Laravel نظام التوجيه في

لماذا التوجيه مهم؟

- بالمنطق المناسب URLs ينظم طلبات التطبيق: يربط □
- للحماية middleware يوفر الأمان: يمكنك إضافة □
- يسهل الصيانة: كل المسارات في مكان واحد □
- احترافية APIs بناء RESTful APIs يدعم □

أين توجد المسارات؟

```
routes/
├── web.php      # مسارات الويب (صفحات HTML)
├── api.php      # مسارات API (JSON responses)
├── console.php  # أوامر Artisan
└── channels.php # Broadcasting channels
```

HTTP أنواع المسارات

الأساسية HTTP Methods يدعم جميع أنواع Laravel.

1. GET - قراءة البيانات

(يُستخدم لعرض البيانات فقط (لا يغير شيئاً في قاعدة البيانات).

```
Route::get('/users', function () {  
    return 'قائمة المستخدمين';  
});  
  
Route::get('/user/{id}', function ($id) {  
    return "عرض المستخدم رقم: $id";  
});
```

متى تستخدمه؟

- عرض صفحة
- قراءة بيانات
- البحث

2. POST - إنشاء بيانات جديدة

يُستخدم لإرسال بيانات جديدة للخادم.

```
Route::post('/users', function () {  
    // إنشاء مستخدم جديد  
    return 'تم إنشاء المستخدم';  
});
```

متى تستخدمه؟

- (Form) إرسال نموذج
- إنشاء سجل جديد
- تحميل ملف

3. PUT/PATCH - تحديث بيانات موجودة

```
// PUT - كامل  
Route::put('/users/{id}', function ($id) {
```

```

    return "بالكامل $id تحديث المستخدم";
  });

// PATCH - تحديث جزئي
Route::patch('/users/{id}', function ($id) {
    return "تحديث بعض بيانات المستخدم $id";
});

```

PUT و PATCH: الفرق بين

- **PUT:** تحديث كامل (كل الحقول)
- **PATCH:** تحديث جزئي (بعض الحقول)

4. DELETE - حذف بيانات

```

Route::delete('/users/{id}', function ($id) {
    return "حذف المستخدم $id";
});

```

5. Methods مسار يقبل عدة

```

Route::match(['get', 'post'], '/form', function () {
    return 'GET و POST يقبل';
});

// جميع الـ Methods
Route::any('/test', function () {
    return 'method يقبل أي';
});

```

Methods: جدول ملخص

Method	الاستخدام	مثال
GET	قراءة/عرض	عرض قائمة المنتجات
POST	إنشاء جديد	إضافة منتج جديد
PUT	تحديث كامل	تحديث كل بيانات منتج
PATCH	تحديث جزئي	تحديث سعر منتج فقط
DELETE	حذف	حذف منتج

معاملات المسار

URL. معاملات المسار تسمح لك بالتقاط قيم من الـ

1. معاملات إلزامية

```
// معامل واحد
Route::get('/user/{id}', function ($id) {
    return "المستخدم رقم: $id";
});

// عدة معاملات
Route::get('/post/{postId}/comment/{commentId}', function ($postId, $commentId) {
    return "المقال $postId - التعليق $commentId";
});
```

URLs: أمثلة

- /user/5 → \$id = 5
- /post/10/comment/3 → \$postId = 10, \$commentId = 3

2. معاملات اختيارية

```
Route::get('/user/{name?}', function ($name = 'ضيف') {
    return "مرحباً $name";
});
```

أمثلة:

- /user/أحمد "مرحباً أحمد"
- /user "مرحباً ضيف"

3. قيود على المعاملات (Regular Expressions)

```
// أرقام فقط
Route::get('/user/{id}', function ($id) {
    return "المستخدم: $id";
})->where('id', '[0-9]+');
```

```
// حروف فقط
Route::get('/user/{name}', function ($name) {
    return "المستخدم: $name";
})->where('name', '[A-Za-z]+' );

// عدة قيود
Route::get('/user/{id}/{name}', function ($id, $name) {
    return "ID: $id, الاسم: $name";
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

4. قيود عامة (Global Constraints)

في `app/Providers/RouteServiceProvider.php`:

```
public function boot(): void
{
    // يجب أن يكون رقم {id} كل
    Route::pattern('id', '[0-9]+' );

    // يجب أن يكون حروف وأرقام {slug} كل
    Route::pattern('slug', '[a-z0-9-]+' );
}
```

المسارات المسماة

المسارات المسماة تسهل الإشارة إلى المسارات في التطبيق.

لماذا المسارات المسماة؟

تجنب الأخطاء: لا داعي ☐ URLs دون تغيير الكود ☒ وضوح الكود: أسماء واضحة بدلاً من URL سهلة التعديل: غير الـ ☐ يدوياً URLs لكتابة

تعريف مسار مسمى

```
Route::get('/user/profile', function () {
    return 'صفحة الملف الشخصي';
})->name('profile');

Route::get('/dashboard', function () {
    return 'لوحة التحكم';
})->name('dashboard');
```

```
})->name('dashboard');
```

استخدام المسارات المسماة

في Blade Templates:

```
<!-- رابط بسيط -->
<a href="{ route('profile') }}">الملف الشخصي</a>

<!-- رابط مع معاملات -->
<a href="{ route('user.show', ['id' => 1]) }}">عرض المستخدم</a>

<!-- Redirect -->
return redirect()->route('dashboard');
```

في Controllers:

```
// Redirect
return redirect()->route('profile');

// معاملات
return redirect()->route('user.show', ['id' => $userId]);

// رسالة
return redirect()->route('dashboard')
    ->with('success', 'تم بنجاح');
```

للمسار URL الحصول على:

```
$url = route('profile'); // http://yourapp.com/user/profile
```

تسمية المسارات بشكل منظم

```
// تسمية واضحة
Route::get('/posts', [PostController::class, 'index'])->name('posts.index');
Route::get('/posts/{id}', [PostController::class, 'show'])->name('posts.show');
Route::post('/posts', [PostController::class, 'store'])->name('posts.store');
Route::put('/posts/{id}', [PostController::class, 'update'])->name('posts.update');
Route::delete('/posts/{id}', [PostController::class, 'destroy'])->name('posts.destroy');
```

نمط التسمية الموصى به: `resource.action`

مجموعات المسارات

مجموعات المسارات تساعد في تنظيم المسارات المتشابهة.

1. Route Prefix

```
// بدون مجموعة
Route::get('/admin/users', function () { });
Route::get('/admin/posts', function () { });
Route::get('/admin/settings', function () { });

// مع مجموعة - أفضل!
Route::prefix('admin')->group(function () {
    Route::get('/users', function () { });           // /admin/users
    Route::get('/posts', function () { });           // /admin/posts
    Route::get('/settings', function () { });        // /admin/settings
});
```

2. Route Middleware

```
Route::middleware(['auth'])->group(function () {
    Route::get('/dashboard', function () {
        return 'الوحة التحكم';
    });

    Route::get('/profile', function () {
        return 'الملف الشخصي';
    });
});
```

3. Name Prefix

```
Route::name('admin.')->group(function () {
    Route::get('/dashboard', function () {
        // ...
    })->name('dashboard'); // الاسم الكامل: admin.dashboard

    Route::get('/users', function () {
        // ...
    })->name('users');      // الاسم الكامل: admin.users
```

```
});
```

دمج كل الخصائص 4.

```
Route::prefix('admin')
    ->middleware(['auth', 'admin'])
    ->name('admin.')
    ->group(function () {

        Route::get('/dashboard', function () {
            return 'الوحة التحكم';
        })->name('dashboard'); // admin.dashboard

        Route::get('/users', function () {
            return 'المستخدمون';
        })->name('users');      // admin.users

        Route::get('/posts', function () {
            return 'المقالات';
        })->name('posts');      // admin.posts
    });
```

النتيجة:

- URL: /admin/dashboard, /admin/users, /admin/posts
- Names: admin.dashboard, admin.users, admin.posts
- Middleware: auth, admin على كل المسارات

Route Model Binding

يجلب النماذج تلقائياً من قاعدة البيانات Laravel ميزة قوية تسمح لـ

1. Implicit Binding (تلقائي)

```
use App\Models\User;

// بدون Model Binding
Route::get('/user/{id}', function ($id) {
    $user = User::findOrFail($id);
    return view('user.profile', ['user' => $user]);
});
```



```
// أبسط - Model Binding مع
Route::get('/user/{user}', function (User $user) {
    // Laravel تلقائياً يستخدم
    return view('user.profile', ['user' => $user]);
});
```

كيف يعمل؟

1. {user} يلاحظ أن المعامل اسمه Laravel
2. User \$user ويلاحظ أن النوع هو
3. تلقائياً User::find(\$id) يبحث عن
4. إذا لم يجد، يرجع 404.

2. Custom Key

لكن يمكنك تغيير ذلك id. يبحث به Laravel، افتراضياً

```
// بدلاً من slug البحث بـ id
Route::get('/post/{post:slug}', function (Post $post) {
    return view('post.show', ['post' => $post]);
});
```

مثال URL: /post/laravel-routing-tutorial

3. تخصيص Model في

في App\Models\Post.php:

```
public function getRouteKeyName()
{
    return 'slug'; // بدلاً من id استخدم slug
}
```

الآن يمكنك:

```
Route::get('/post/{post}', function (Post $post) {
    // تلقائياً slug يبحث بـ
    return view('post.show', ['post' => $post]);
});
```

إعادة التوجيه (Redirects)

أنواع Redirects:

```
// 1. Redirect بسيط
Route::get('/old-page', function () {
    return redirect('/new-page');
});

// 2. Redirect إلى مسار مسمى
Route::get('/home', function () {
    return redirect()->route('dashboard');
});

// 3. Redirect 301 دائم
Route::redirect('/old', '/new', 301);

// 4. Redirect للخلف
return back();

// 5. Redirect مع بيانات
return redirect()->route('dashboard')
    ->with('success', 'لتم بنجاح');
```

Fallback Route

مسار احتياطي عند عدم تطابق أي مسار:

```
Route::fallback(function () {
    return view('errors.404');
});
```

عرض جميع المسارات

```
php artisan route:list
```

خيارات مفيدة:

```
# مسارات معينة فقط
php artisan route:list --path=api

# بحث
php artisan route:list --name=user

# مع middleware
php artisan route:list --middleware=auth
```

أفضل الممارسات

أفعل: 

1. استخدم أسماء واضحة

```
Route::get('/posts', [PostController::class,
'index'])->name('posts.index');
```

2. نظم المسارات في مجموعات

```
Route::prefix('admin')->group(function () {
    // مسارات الأدمن
});
```

3. استخدم **Resource Routes** للـ **CRUD**

```
Route::resource('posts', PostController::class);
```

4. Model Binding استخدم

```
Route::get('/user/{user}', function (User $user) {  
    return view('user', compact('user'));  
});
```

❓ لا تفعل:

1. routes في logic لا تضع

```
// سيء ❌  
Route::get('/users', function () {  
    $users = User::all();  
    // سطر من الكود... 50  
};  
  
// جيد ✅  
Route::get('/users', [UserController::class, 'index']);
```

2. لا تكرر نفس الكود

```
// سيء ❌  
Route::get('/admin/users', ...)->middleware('auth');  
Route::get('/admin/posts', ...)->middleware('auth');  
  
// جيد ✅  
Route::middleware('auth')->prefix('admin')->group(...);
```

التمارين العملية

❓ التمرين 1: مسارات أساسية

routes/web.php أنشئ المسارات التالية في

```
// صفحة رئيسية 1.  
Route::get('/', function () {
```

```

        return view('welcome');
    });

    // صفحة "عنا" 2.
    Route::get('/about', function () {
        return view('about');
    });

    // صفحة اتصل بنا 3.
    Route::get('/contact', function () {
        return view('contact');
    });

```

التمرين 2: معاملات المسار

```

// 1. عرض منتج بـ ID
Route::get('/product/{id}', function ($id) {
    return "عرض المنتج رقم: $id";
})->where('id', '[0-9]+');

// 2. عرض منتج بـ slug
Route::get('/product/{slug}', function ($slug) {
    return "عرض المنتج: $slug";
})->where('slug', '[a-z0-9-]+');

// 3. معاملات متعددة
Route::get('/category/{category}/product/{product}', function ($category, $product)
{
    return "التصنيف: $category - المنتج: $product";
});

```

التمرين 3: المسارات المسماة

```

Route::get('/dashboard', function () {
    return view('dashboard');
})->name('dashboard');

Route::get('/profile', function () {
    return view('profile');
})->name('profile');

// في Blade:
// <a href="{ {{ route('dashboard') }}">لوحة التحكم</a>

```

التمرين 4: مجموعات المسارات

```
// مجموعة مسارات الأدمين
Route::prefix('admin')->name('admin.')->middleware('auth')->group(function () {
    Route::get('/dashboard', function () {
        return 'لوحة تحكم الأدمين';
    })->name('dashboard');

    Route::get('/users', function () {
        return 'إدارة المستخدمين';
    })->name('users');

    Route::get('/posts', function () {
        return 'إدارة المقالات';
    })->name('posts');
});
```

المختلفة HTTP Methods التمرين 5: أنواع

```
// نموذج اتصل بنا
Route::get('/contact', function () {
    return view('contact');
})->name('contact.show');

Route::post('/contact', function () {
    // معالجة البيانات
    return redirect()->route('contact.show')
        ->with('success', '!تم إرسال رسالتك');
})->name('contact.submit');
```

كامل CRUD التحدي: نظام

```
// Posts CRUD
Route::get('/posts', function () {
    return 'قائمة المقالات';
})->name('posts.index');

Route::get('/posts/create', function () {
    return 'إضافة مقال جديد';
})->name('posts.create');

Route::post('/posts', function () {
    return 'حفظ المقال';
})->name('posts.store');
```

```
Route::get('/posts/{id}', function ($id) {
    return "عرض المقال $id";
})->name('posts.show');

Route::get('/posts/{id}/edit', function ($id) {
    return "تعديل المقال $id";
})->name('posts.edit');

Route::put('/posts/{id}', function ($id) {
    return "تحديث المقال $id";
})->name('posts.update');

Route::delete('/posts/{id}', function ($id) {
    return "حذف المقال $id";
})->name('posts.destroy');
```

الملخص ؟

في هذا الدرس، تعلمت:

معاملات المسار (إلزامية واختيارية) ؟ قيود المعاملات ؟ HTTP Methods (GET, POST, PUT, DELETE) أنواع ؟
Route Model Binding ؟ المسارات المسماة وفوائدها ؟ مجموعات المسارات وتنظيمها ؟ (Regular Expressions)
أفضل ممارسات التوجيه ؟

موارد إضافية ؟

- [Laravel Routing Documentation](#)
- [RESTful Resource Controllers](#)
- [Route Model Binding](#)

اختبر نفسك ؟

قبل الانتقال إلى الدرس 3، تأكد من قدرتك على:

1. GET و POST ما الفرق بين ؟

2. كيف تنشئ معامل مسار اختياري؟
 3. لماذا نستخدم المسارات المسماة؟
 4. prefix؟ كيف تنشئ مجموعة مسارات بـ
 5. Route Model Binding ما هو
-

الدرس التالي

MVC مستعد للمزيد؟ انتقل إلى **الدرس 3: المتحكمات ونمط**

في الدرس 3، ستتعلم

- إنشاء Controllers
 - تنظيم منطق التطبيق
 - Resource Controllers
 - Dependency Injection
 - والمزيد!
-

👉 **تعلم سعيد**